

Правила разработки больших проектов

> **Назначение:** методология ведения объёмных задач с использованием планов, поэтапной фиксации, баз знаний и стратегии экономии токенов. Шаблон для переноса в другие проекты.

1. Классификация задач

1.1. Короткая задача

Решается сразу, без имени и плана. Признаки:

- один объект или быстрый ответ;
- менее 3 этапов;
- менее 3 объектов кода.

1.2. Объёмная задача

Признаки (достаточно любого):

- пользователь назвал её задачей или дал имя (ключ из трекера);
- ожидается более 3 самостоятельных этапов;
- затрагивает более 3 объектов кода или несколько папок.

Объёмная задача → создать уникальное имя и общий план. Далее работать только по плану.

2. Структура папки задачи

Каждая задача получает собственную папку:

```
<TaskRoot>/<TaskName>/  
  
├─ Git_YYYY_MM_DD/           # Оригиналы объектов (для сравнения)  
  
├─ Test_YYYY_MM_DD/          # Готовые объекты для тестирования  
  
├─ DIAG_YYYY_MM_DD_hh_mm/    # Объекты для диагностики  
  
├─ Ready_YYYY_MM_DD/         # Готовые (после тестирования)  
  
├─ Read_YYYY_MM_DD/          # Готовые к выводу в продакшн  
  
└─ Describe/                 # Анализы, план, описания изменений  
  
    └─ ПЛАН_РЕАЛИЗАЦИИ.txt  
  
    └─ <объект1>_анализ.txt  
  
    └─ <объект2>_изменения.txt
```

Правила именования

- Test_YYYY_MM_DD__hh_mm — если папка за дату уже существует, добавить суффикс времени.
- DIAG_YYYY_MM_DD_hh_mm — всегда с датой и временем (итерации диагностики).
- Суффикс _<число> = ключ родительской задачи (если ТЗ в родительской задаче).

3. План реализации (ПЛАН_РЕАЛИЗАЦИИ.txt)

Обязательные разделы:

ЦЕЛЬ: <описание цели>

СТАТУС: в работе / отложено / завершено

ЭТАПЫ:

[] Этап 1 – описание

[~] Этап 2 – описание (в работе)

[x] Этап 3 – описание (готов, результат)

[!] Этап 4 – описание (отложен на дорогую модель)

СЛЕДУЮЩИЙ ШАГ: <с чего продолжить>

ОЧЕРЕДЬ ДОРОГИХ ПОДЗАДАЧ:

- Этап 4: нужен мощный LLM для <обоснование>

Статусы этапов

- [] — не начат
- [~] — в работе
- [x] — готов
- [!] — отложен на дорогую модель

Дублирование статуса

- Сводная запись в базе знаний (одна запись на задачу, не дробить).
- Статический якорь в `pending-tasks.md` (виден без баз знаний).

4. Поэтапное выполнение

4.1. Перед этапом

1. Пометить [~] в плане.
2. Синхронизировать список задач сессии.

4.2. После этапа

1. Пометить [x] в плане.
2. Записать результат в план и `Describe/<объект>.txt` .
3. Обновить сводную запись в базе знаний.

4.3. Фиксация после КАЖДОГО этапа

Чтобы любая следующая сессия продолжилась без потерь:

- результат этапа;
- обновлённый «СЛЕДУЮЩИЙ ШАГ»;
- актуальный статус плана.

5. Стратегия экономии токенов (дорогие vs дешёвые модели)

5.1. Роутинг по стоимости

- Основная обработка по плану → дешёвая/бесплатная модель.
- Дорогая модель → только этапы [!] и только с готовым брифом.

5.2. Истощение ресурса модели

Признаки: ошибки баланса, лимит контекста, деградация ответов.

Порядок:

1. Немедленно зафиксировать текущий этап ([~] → результат + СЛЕДУЮЩИЙ ШАГ) в плане, базе знаний, `pending-tasks.md`.
2. Не начинать новый этап на истощённом ресурсе.
3. Новая модель возобновляет по явной команде пользователя.

5.3. Подготовка перед запуском дорогой модели

Дешёвая модель обязана выполнить всё возможное:

- Собрать точный контекст: нужные фрагменты кода (не целые файлы), карту зависимостей, релевантные знания.
- Сузить задачу до конкретного вопроса/объекта.
- Сформулировать ожидаемый результат и критерий приёмки.
- Выполнить черновой/частичный вариант, если возможно.
- Готовый «пакет» сохранить в `Describe/<имя>_SUB_<n>_бриф.txt` .

6. Базы знаний

6.1. Разделение по назначению

База	Что сохранять
Проектная (ais-catalog)	Объекты, бизнес-логика, связи, задачи, особенности обработки
Инструментальная (opencode-mcp)	Скрипты, настройки, ошибки конфигурации, опыт работы с инструментом

6.2. Дисциплина записей

- **Писать сводную запись**, не дробить. Одна запись на задачу со структурой: анализ → вывод → карта воздействия → точка возобновления.
- **Явные теги** (имя задачи, имя объекта) — работают точнее FTS-поиска.
- **Проверять дубли** перед сохранением.

6.3. Knowledge First (знания в первую очередь)

При любом не-простом запросе:

1. Поиск в базах знаний.
2. Если найден ответ — использовать его.
3. Если не найден — самостоятельный анализ.
4. Если решение новое — сохранить в базу.

7. Остановка и продолжение задачи

7.1. Остановка

Не продолжать самостоятельно, если следующий шаг требует решения аналитика.

Сохранить:

- Сводная запись в базе знаний (анализ, вывод, карта воздействия, точка возобновления).
- Файлы задачи в `Describe/` (анализ + план).
- Статический якорь в `pending-tasks.md`.

7.2. Продолжение (только по явной команде)

1. Поиск в базе знаний по тегам задачи.
2. Прочитать `Describe/` и план.
3. Продолжить с раздела «СЛЕДУЮЩИЙ ШАГ».

8. Параллельная работа над несколькими задачами

1. Каждая задача — в своей папке и своей сессии.
2. В одной сессии не ведутся две задачи с разными планами.
3. Базы знаний разделяются по назначению (проектная vs инструментальная).
4. При исчерпании ресурса — зафиксировать этап и продолжить на другой модели.
5. Сессии именуются по шаблону `<ключ> YYYY-MM-DD HH:MM` для различения повторных запусков.

9. Правило «ЗАПОМНИ» — куда писать правила

Тип правила	Куда писать
Правило кодировки, безопасности	Файл <code>AGENTS.md</code> или <code>preflight.mdc</code>

Особенность объекта кода	Файл <code>pb-object-rules.mdc</code> / <code>sql-rules.mdc</code>
Ошибка + решение (баг)	Файл <code>changelog.md</code>
Термин / сокращение	Файл <code>abbreviations.md</code> / <code>glossary.mdc</code>
Настройка инструмента	Знание в инструментальной базе
Опыт работы с задачей	Знание в проектной базе

Критерий: если правило должны знать все будущие сессии → файл. Если опыт конкретной ситуации → знание в базе.

10. Кодировки (обязательно)

Тип файла	Кодировка	BOM
<code>.ps1</code>	UTF-8	с BOM
<code>.json</code> / <code>.jsonc</code>	UTF-8	без BOM
<code>.sql</code>	Windows-1251	нет
<code>.bat</code>	Windows-1251	нет
<code>.md</code>	UTF-8	без BOM
<code>.sr*</code> (экспорт)	Windows-1251	нет

11. Безопасность

- Секреты (логины, пароли, токены) — только в `.local_secrets.json`, никогда в Git.
- Исполняемые модули читают секреты из `.local_secrets.json` или как параметры.
- При диагностических SQL-запросах **не выводить** конфиденциальные данные (имена, суммы, адреса). Выводить только ID, коды, статусы.

12. Документация — тройной формат

При создании документации `.md` одновременно формировать `.html` и `.pdf`:

Требование	Значение
Язык	Только русский, подробно
Фон	Белый или светло-серый (<code>#F5F7FA</code>)
Цвет текста	Тёмный (<code>#1A3A60</code> , <code>#2D3748</code>)
Шрифт	Системный с поддержкой кириллицы (Segoe UI, Arial)
Полные пути	Указывать полные пути, не относительные

13. Чек-листы

13.1. Запуск новой задачи

- [] Имя задачи распознано.
- [] Найден/создан корневой путь задачи.
- [] Проверено наличие папки `<имя>_*`.
- [] Создана структура: `Git_`, `Test_`, `Describe/`.
- [] Выгружены оригиналы.
- [] Создан план реализации с «СЛЕДУЮЩИМ ШАГОМ».
- [] План продублирован в базе знаний и `pending-tasks.md`.

13.2. Продолжение задачи

- [] Получена явная команда «продолжить <имя>».
- [] Выполнен поиск в базе знаний.
- [] Прочитаны `Describe/` и план.
- [] Определён «СЛЕДУЮЩИЙ ШАГ».
- [] Работа идёт только в папке задачи.

13.3. Остановка задачи

- [] Зафиксирован результат текущего этапа.
- [] Обновлён план (статусы, «СЛЕДУЮЩИЙ ШАГ»).
- [] Сохранена сводная запись в базе знаний.
- [] Обновлён `pending-tasks.md`.

14. Адаптация под проект

При переносе в другой проект:

1. Определите корневой путь задач (`<TaskRoot>`).
2. Настройте имена баз знаний под свою инфраструктуру.
3. Адаптируйте правило TASK под свой трекер задач (Jira, YouTrack, GitHub Issues).
4. Настройте кодировки под свою среду разработки.
5. Определите, какие правила должны быть статическими (файлы), а какие — динамическими (базы знаний).